2025-11-09

package cpen221.graphs.graph; import java.util.List; import java.util.Map; import java.util.Set; // TODO: Add documentation (Task 1) public interface Graph<V extends Vertex, E extends Edge<V>> { /* === TASK 1 === */ boolean hasVertex(V v); boolean hasEdge(E e); boolean hasEdge(V v1, V v2); double edgeLength(V v1, V v2); /* === TASK 2 === */ double edgeLengthSum(); E getEdge(V v1, V v2); Set<V> getVertices(); Set<E> getEdges(); Set<E> getEdges(V v); Map<V, E> getNeighbours(V v); /* === TASK 3 === */ double pathCost(List<V> path, PathCostType costType); List<V> minimumCostPath(V source, V sink, PathCostType costType); /* === TASK 4 === */ double getDiameter(PathCostType costType); V getCenter(PathCostType costType); /* === TASK 5 === */ Set<Set<V>> getComponents(); } what is the use of generics here? what would V and E be? can they be anything? does it actually have to be specificed? or is it just an object of type <type>

ChatGPT said:
Thought for 14s

Nice, welcome to Java generics land 😌

Let's walk through this line:

```
public interface Graph<V extends Vertex, E extends Edge<V>>
```

1. What's the point of generics here?

Generics let this `Graph` interface work with *any* kind of vertex and edge **while still being type-safe**.

So instead of everything being just `Vertex` and `Edge` (and you casting all over the place), you say:

- "This graph is specifically made of **this kind of vertex** (`V`)"
- "And **this kind of edge** (`E`), which connects those vertices"

Then all the methods can use `V` and `E` consistently:

```
Set<V> getVertices();    // not Set<Vertex>
Set<E> getEdges();       // not Set<Edge>
Map<V, E> getNeighbours(V v);
```

No casting, fewer runtime surprises.

---

2. What exactly are `V` and `E`?

They're **type parameters** with bounds:

```
V extends Vertex
E extends Edge<V>
```

So:

- `V` must be some class that implements/extends `Vertex`
- `E` must be some class that implements/extends `Edge<V>` (i.e., an edge whose endpoints are of type `V`)

Example:

```
class City implements Vertex { ... }
class Road extends Edge<City> { ... }

Graph<City, Road> g = ...;
```

Now:

- `getVertices()` returns `Set<City>`
- `getEdges()` returns `Set<Road>`
- `getNeighbours(City v)` uses `City`, not some generic `Vertex`

So `V` and `E` are **not literally "anything"** — they must respect those `extends` bounds.

---

3. Do they *have* to be specified?

When you **implement** or **use** the interface in real code: yes.

Two common patterns:

Keep it generic in your implementation:

```
public class AdjacencyListGraph<V extends Vertex, E extends Edge<V>>
        implements Graph<V, E> {
    ...
}
```

Then later:

```
Graph<City, Road> g = new AdjacencyListGraph<>();
```

1.

Or lock it to concrete types:

```
public class CityRoadGraph implements Graph<City, Road> {
    ...
}
```

2.

Using `Graph` *without* type arguments:

```
Graph g = ...;   // raw type
```

is legal but gives compiler warnings and throws away type safety. CPEN 221 profs would give you The Look™ for that.

---

4. “Is it just an object of type `<type>`?”

Kind of:

- At **compile time**, `Graph<City, Road>` and `Graph<UserNode, FriendshipEdge>` are treated as different typed things.
- At **runtime**, Java erases generic type info (type erasure), so it’s just `Graph` under the hood — but the compiler has already enforced all the type safety.

So: generics here are mainly a compile-time guarantee that your graph is internally consistent: all edges connect the right kind of vertices, and all your sets/maps/lists use the right types.

If you want, you can paste your `Vertex` and `Edge` interfaces and we can walk through a concrete “this is what V/E *actually* are in this assignment” example.

You said:
so our V and E here dont mean anyhting yet, they arnt declared as a variable yet, butwhen we create graphs they have to be safe versionsn that implement or extend the correct classes, which ensures type safety?

ChatGPT said:
Thought for a few seconds

Yeah, you’ve basically got it — just need to tighten the picture a bit.

In the interface:

```
public interface Graph<V extends Vertex, E extends Edge<V>> { ... }
```

- `V` and `E` are **type parameters**, not variables. They don’t “exist” as real types yet — they’re placeholders with rules:
    - `V` must be some subtype of `Vertex`
    - `E` must be some subtype of `Edge<V>`

When you actually *use* the interface, you plug in real types:

```
class City implements Vertex { ... }
class Road extends Edge<City> { ... }

Graph<City, Road> g = new AdjacencyListGraph<>();
```

- Now:
    - `V` is `City`
    - `E` is `Road`

    - The compiler enforces that everywhere in this graph, vertices are `City` and edges are `Road`, and that `Road` really is an `Edge<City>`.

So:

> "V and E here don't mean anything yet"

Correct in the sense that they're not concrete classes yet — they're placeholders.

> "when we create graphs they have to be safe versions that implement or extend the correct classes, which ensures type safety?"

Yup. When you say `Graph<SomeVertexType, SomeEdgeType>`, the compiler checks:

- `SomeVertexType extends Vertex`
- `SomeEdgeType extends Edge<SomeVertexType>`

If that doesn't hold, it just won't compile. That's the whole type-safety win.

You said:
so we would still have to declare like a class city and declare a class Road, its just like another safety/firewall to prevent us fro putting in a different type of thing. but the good part is, it doesnt have to be just a city, it could be a rural city that implements vertex, or towns implements vertex, etc

ChatGPT said:

Exactly, you're seeing it clearly now.

You **still** write concrete classes like:

```
class City implements Vertex { ... }
class Road extends Edge<City> { ... }
```

-
- The generics on `Graph<V, E>` just say:

> "Whatever you use for `V` must be a `Vertex`, and whatever you use for `E` must be an `Edge` of that same `V`."

So later you can do:

```
class RuralCity implements Vertex { ... }
class Town implements Vertex { ... }
class Road extends Edge<RuralCity> { ... }

Graph<RuralCity, Road> ruralGraph = new AdjacencyListGraph<>();
Graph<Town, Edge<Town>> townGraph = new AdjacencyListGraph<>();
```

Each `Graph<..., ...>` is now **locked** to those specific types. You can't accidentally mix a `Town` into a `Graph<City, Road>` or use an `Edge<Town>` in a graph of `City` vertices — the compiler stops you.

So yeah:

- Generics = reusable graph abstraction
- Bounds (`extends Vertex`, `extends Edge<V>`) = firewall so only "legal" vertex/edge types get in
- And "legal" can be `City`, `RuralCity`, `Town`, or whatever, as long as they implement `Vertex` (and the edges match).

2025-11-14
Alright boy its time for task 5, your going to be doing moost of the heavy lifting here unfortunately.. What the fuck is he saying

ChatGPT said:
Thought for 2m 41s

Yeah this write-up is… a lot of words for something pretty simple 😅
Let me translate it into "Ethan English".

---

1. What are we even doing?

You have a weighted, **undirected** graph.
You want to split its vertices into `numberOfPartitions` **clusters** (they call them "partitions" or "components").

Then, for each cluster of vertices, you return the **induced subgraph**:

> same vertices, and *all* edges from the original graph whose endpoints are both inside that cluster.

So `partitionGraph(g, k)` → a list/collection of `k` subgraphs.

---

2. The two weird conditions

They're basically saying: *clusters should group together vertices that are close in edge length.*

Imagine edge lengths are distances: shorter = more similar.

(a) Nearest-Neighbour Condition

Take any vertex `v` that is in a cluster with at least one other vertex.

Look at all edges coming out of `v`.
Find the **shortest** ones (maybe there are ties).

> The rule: **at least one** of the vertices connected to `v` by a shortest edge must be in the **same cluster** as `v`.

Rephrase:

> If `v` is grouped with others, then one of its closest neighbours must be in that same group.
> `v` is not "closer" to someone outside the group than to everyone inside.

If `v` is in a singleton partition `{v}`, this rule doesn't apply.

(b) Shortest-Edge Condition

Now look at the **whole graph** and its edges.

Sort all edges by length, from shortest to longest.

> The endpoints of the **shortest edges** are **not allowed to end up in different partitions**.
> If an edge `(u, v)` is very short, then `u` and `v` must be in the same partition.

Their more formal wording is: if you're relying on some edge (v_1, v_2) of length L to justify putting vertices together, then **any edge shorter than L** must also have both endpoints in the same partition. In plain terms:

> You're only allowed to "cut" relatively *long* edges.
> All the *really* short edges must stay inside some partition.

---

3. The example graph

Edges (with lengths):

- A–B = 5
- A–C = 10
- B–C = 3
- C–D = 1 (shortest)

Sorted edges: CD(1), BC(3), AB(5), AC(10)

Four partitions

- {A}, {B}, {C}, {D}
- Trivial: each vertex alone, no rules violated.

Three partitions

We must merge exactly two vertices (because we start with 4 singletons).

Shortest edge is C–D with length 1.

- Shortest-edge rule: C and D **must** be together.
- So we get {C,D}, and A and B stay alone: {A}, {B}, {C,D}.

Any other pair (like `{A,B}`) would leave `C` and `D` separated and would **break** the shortest-edge rule.

Two partitions

Start from singletons and use the "shortest edges first" idea:

1. Use edge `C–D (1)` → merge `{C}` and `{D}` → `{C,D}`, `{A}`, `{B}`
2. Next edge `B–C (3)` → merges `{B}` with `{C,D}` → `{B,C,D}`, `{A}`

We now have 2 partitions: `{A}` and `{B,C,D}`.

Is `{A,B}` + `{C,D}` allowed?
No, because of the **nearest-neighbour** rule:

- For vertex `B`: edges are `B–A = 5`, `B–C = 3`.
- Its closest neighbour is `C`, but in `{A,B}` + `{C,D}`, `B` is grouped with `A`, not `C`.
  So `B` is closer to someone **outside** its partition → rule broken.

One partition

Just put everything together: `{A,B,C,D}` (the original graph).

---

4. How to *think* about the algorithm

What they *want* you to implement is basically:

> **Kruskal-style clustering**:
> Start with each vertex alone and keep gluing components together by the shortest edges until you have the requested number of partitions.

Algorithm sketch:

1. **Start**: each vertex is its own partition.
2. **List all edges** with their lengths.

3. **Sort** edges by increasing length.
4. Use a **union-find / disjoint-set** structure (or equivalent bookkeeping of components).
5. For each edge `(u, v)` from shortest to longest:
    - If `u` and `v` are in **different** components **and** you currently have more components than `numberOfPartitions`, merge those two components.
    - Otherwise skip (either already same component, or you've already reached the desired number of partitions).
6. When you have exactly `numberOfPartitions` components, stop.
7. For each component (set of vertices), build the **induced subgraph**: include all edges from the original graph whose endpoints both lie in that component.
8. Return the list of those subgraphs.

Why this automatically satisfies the two rules:

- **Shortest-edge rule**:
  You're literally merging along edges from shortest to longest.
  So endpoints of the globally shortest edges always end up in the same component.
- **Nearest-neighbour rule**:
  For any vertex that gets joined into a multi-vertex component, its **first** merge event uses one of its shortest incident edges (because we process edges in sorted order).
  That neighbour lives in the same component → rule satisfied.

If you're down, next step we can:

- Look at your actual `Graph`/`Edge` interfaces, and
- Write the Java `partitionGraph` skeleton with union-find + the induced-subgraph construction.

You said:

wait so the overall rpocess is pretty simple then no? we first break up the grpah into components, then we start splitting them into partitiions. so in our case we start with each vertex in their own partiitions, then we list all the edges with their lengths, this was done previously, then we sort all of the edges by increasing legnth and then use some kind of structure to bookkeep all of the compoennts. then we check for each edge(u, v), from shortest to longest. if u and v are in different components and you currently have more components than numberOfPartiitions, merge those two components. otherwise we go ahead and skip them, since they are either already in the same component, or we have already reached the desired number of partions required. once we reach the required amount of partiitions, then we can go ahead and stop. then for each of these compoennts, we build the induced subgraph, including all edges from the origonal graph whos endpoints both lie inside of the compoennt. then we return a list including all of these subgraphs, with a key mapping to what partiiton it is, and then the subgraoh.

ChatGPT said:

Thought for 53s

Yup, you've basically got it. 🎓
Let me just clean it up and point out a couple tiny gotchas.

Big picture

What you wrote is **exactly** the Kruskal-style clustering idea they're hinting at. The flow should be:

1. **Start with every vertex in its own partition.**
   (So you already have `|V|` partitions.)

2. **Get all edges and their lengths** (you already have this from earlier tasks).

3. **Sort edges by increasing length.**

4. **Maintain components/partitions** with some bookkeeping structure:

   - Either a proper **union–find (disjoint set)**, or

   - A `Map<V, Integer>` from vertex → partitionId plus a `Map<Integer, Set<V>>` of vertices in each partition (slower but ok for small graphs).

5. **Process edges from shortest to longest:**

For each edge `(u, v)`:

- Find the partitions `Pu`, `Pv` that `u` and `v` are in.
- If `Pu != Pv` **and** you currently have **more partitions than `numberOfPartitions`**, then **merge** those two partitions.
    - Decrease the partition count.
- Otherwise, skip the edge.

6. Stop as soon as `currentPartitionCount == numberOfPartitions`.

   This is what enforces:

   - **Shortest-edge condition**: you only ever "cut" long edges; all shorter edges have already been used to merge things.
   - **Nearest-neighbour condition**: the *first* time a vertex joins a multi-vertex partition, it must be via one of its shortest edges (because edges are processed in global sorted order).

7. **Build induced subgraphs for each partition:**

   For each partition `Pi` (a set of vertices):

   - Create a new empty graph `Gi`.
   - Add all vertices in `Pi` to `Gi`.
   - For every original edge `(x, y)`:
       - If **both** `x` and `y` are in `Pi`, add that edge to `Gi`.

8. Collect all these `Gi` into a `List<Graph<...>>` or whatever the spec wants.

   You don't *need* a fancy "key → subgraph" mapping unless the method's return type forces it; a list of subgraphs is usually enough (their order can be arbitrary or by partitionId).

Minor clarifications

- You said: “we first break up the graph into components, then we start splitting them into partitions.”
  Careful with wording: what we’re actually doing is

  start with max partitions and merge until we hit the requested number.

  The `getComponents()` method is a *separate* task: normal connected-component detection. You don’t strictly need it inside `partitionGraph` if you just run the shortest-edge merging on the whole graph.

- Disconnected graphs:
  This algorithm never merges across disconnected components anyway (no edges), so connected components stay separate unless you intentionally force-merge without edges (which we *don’t* do here).

So yes: the process really is that simple. Your summary is basically correct — just think “start with each vertex alone, then merge along shorter and shorter edges until you have `k` blobs, then spit out induced subgraphs for each blob.”

If you want, next step we can sketch actual Java for:

- the bookkeeping structure, and
- the “build induced subgraphs” part that uses your `Graph` / `Edge` interfaces.

You said:
wouldnt this be very very slow though if we were working with a data set with afew thousand to millions of vertices and edges

ChatGPT said:
Thought for 9s

Good instinct to worry about that, but this particular approach is actually **pretty much as good as it gets** for large graphs—as long as we implement it the right way.

---

1. Complexity of the algorithm we described

Using a **union–find** / disjoint-set structure:

1. **Get all edges**:
   You already have them → `O(E)`.

2. **Sort edges by length**:
   `O(E log E)` time.

3. **Process edges with union–find**:

   - Each edge: 2 finds + maybe 1 union.

   - Amortized cost: almost `O(1)` per operation (`α(V)` inverse Ackermann).

4. So this step is `O(E α(V)) ≈ O(E)`.

5. **Build induced subgraphs**:

   - Visit each vertex once: `O(V)`.

   - Visit each edge once to check endpoints: `O(E)`.

Total: **`O(E log E + V)`** → **dominated by `O(E log E)`**.

Space: `O(V + E)` for the graph plus `O(V)` for union–find & partitions.

---

2. Is that “slow” for thousands / millions?

- **Thousands of vertices/edges** → absolutely trivial.

- **Millions of edges**:

  - `E log E` with `E = 10^6` is around `10^6 * 20` operations-ish.
    In Java/C++ that’s totally reasonable.

  - This is literally the same complexity as Kruskal’s MST algorithm, which *is* used on million-edge graphs in practice.

Also note: any algorithm that looks at **all edges** must at least be `Ω(E)` just to read the input. Sorting all of them adds only a `log E` factor. So we're not doing anything ridiculous.

---

3. When would it be "very slow"?

Only if we implement the bookkeeping in a **naive** way, like:

- Keeping partitions as `List<Set<V>>` and, for each edge, linearly searching which set contains `u` and `v`.
  That can blow up to something like `O(E * V)` → *that* is bad for large graphs.

So for performance, the key things are:

- Use **union–find** (or at least a `Map<V, componentId>` and merge IDs smartly).
- Store the graph as an **adjacency list**, not an adjacency matrix, if it's sparse.

For your MP2 test data, this is *way* overkill performance-wise anyway; they'll have pretty small graphs. But even in the "millions of edges" fantasy, the algorithmic strategy is still solid.

If you want, next we can sketch a union–find-based Java implementation that avoids the slow patterns, so you're future-proof even outside the assignment.

shoulld we also be using a check rep function during everysingle exit point? then thats going to mess up our code coverage by alot alot

Yeah, you **don't** want to spam `checkRep` literally on every exit point of every method. That's overkill and not what they're looking for.

Let's separate a few things:

---

1. Where `checkRep` actually makes sense

Use `checkRep` **only in classes that own mutable state**, like:

- `AdjacencyListGraph`
- `AdjacencyMatrixGraph`
- maybe `Edge` / `Vertex` (but they're basically immutable and enforced in constructor anyway)

And even in those, you usually:

- Call `checkRep()`:
    - at the **end of constructors**, and
    - at the **end of mutator methods** (`addVertex`, `addEdge`, `removeVertex`, etc.)
- You **do not** need it in read-only/observer methods (`getVertices`, `getEdges`, `getNeighbours`, etc.).

That's enough to show you're enforcing the RI and that your AF/RI story is real.

---

2. How to wire it without wrecking performance or style

Canonical pattern for this course:

```
private void checkRep() {
    assert adj != null;
    for (var entry : adj.entrySet()) {
        V v = entry.getKey();
        Map<V, E> nbrs = entry.getValue();
        assert nbrs != null;
        for (var e : nbrs.entrySet()) {
            V u = e.getKey();
            Edge<V> edge = e.getValue();
            assert adj.containsKey(u);
            assert edge != null;
            assert edge.incident(v) && edge.incident(u);
        }
```

```
    }
}
```

Then just call:

```
public boolean addVertex(V v) {
    if (v == null || hasVertex(v)) {
        return false;
    }
    adj.put(v, new HashMap<>());
    checkRep();
    return true;
}
```

Because `checkRep` uses **`assert`**, it only runs when tests / runtime are started with `-ea` (enable assertions). If they run JUnit *without* enabling assertions, `checkRep` is basically free.

---

3. Will this murder my coverage?

No, not in any meaningful way.

- Coverage tools count lines you **could** execute.
- `checkRep()` being called just means those lines can get executed in normal runs.
- You are **not** penalized for “too many lines”. You’re penalized if **core logic** isn’t covered.

Worst-case: some `assert` lines in `checkRep` aren’t all triggered by tests (e.g., a rare branch). That’s fine. They care way more about coverage of:

- `addVertex`, `addEdge`, `removeVertex`, `getCenter`, `getDiameter`, `minimumCostPath`, `partitionGraph`, etc.

You absolutely **do not** need to hit every internal assertion in `checkRep` to reach “A-range coverage”.

---

4. Practical recommendation for you

For this MP, I'd do:

- Add a **private `checkRep()`** to:
    - `AdjacencyListGraph`
    - `AdjacencyMatrixGraph`
    - (optionally `Edge`, but your constructor basically enforces RI already)
- Call it at:
    - end of constructors,
    - end of public mutators (`addVertex`, `addEdge`, `removeEdge`, `removeVertex`).
- Don't touch observers with it.
- No need for `checkRep` in `Algorithms` or `GeneralGraph` (they don't own representation).

That's enough to satisfy the "reasonable RI + AF + enforcement" requirement without exploding your test suite or making coverage a nightmare.

If you want, next message I can sketch a minimal `checkRep` for `AdjacencyListGraph` and `AdjacencyMatrixGraph` that matches the RI we already wrote.

alright so go through each file, show me where i should add in check rep for each file, and make sure they are concurrent to the rep invariants that you showed me earlier. we also dont need to check rep for like observers, usually just for creaters, producers, and mutators.

Thought for 20s

Nice, this is the right level of paranoia 😆

We'll only add `checkRep` where it actually makes sense: classes with internal mutable state / non-trivial RI. We'll **call it only from creators and mutators**, not observers.

Below is class-by-class what to add and where.

---

1. `Vertex` – no `checkRep` needed

`Vertex` is:

- Immutable (`id` is `final`)
- RI is trivial: “id is an integer; equals/hashCode based on id”

You don’t need a `checkRep` here. If you *really* want, you could validate in the constructor, but there’s nothing meaningful to assert beyond “object exists”.

So: **no changes needed** for `Vertex`.

---

2. `Edge` – add `checkRep`, call it in constructors

Add this method inside `Edge<V extends Vertex>`

Put this near the top or bottom of the class:

```
   /* Representation invariant checker. Enabled only when assertions
(-ea) are on. */
    private void checkRep() {
        assert v1 != null;
        assert v2 != null;
        assert !v1.equals(v2);
        assert length >= 0;
        assert v1.id() < v2.id();
    }
```

Call it at the end of both constructors

In the 1-arg constructor:

```
    public Edge(V v1, V v2) {
        this(v1, v2, 1);
        // checkRep() already called in the other constructor,
        // so no need to call it again here.
    }
```

So no extra call needed there.

In the 3-arg constructor, add `checkRep();` at the end:

```
    public Edge(V v1, V v2, double length) {
        if (v1 == null || v2 == null) {
            throw new IllegalArgumentException("Vertices cannot be
null");
        }
        if (v1.equals(v2)) {
            throw new IllegalArgumentException("The same vertex cannot
be at both ends of an edge");
        }
        if (length < 0) {
            throw new IllegalArgumentException("Edge weight cannot be
negative");
        }

        if (v1.id() < v2.id()) {
            this.v1 = v1;
            this.v2 = v2;
        }
        else {
            this.v1 = v2;
            this.v2 = v1;
        }

        this.length = length;

        checkRep();
    }
```

RI is enforced both by explicit checks + `checkRep` asserts.

---

3. `AdjacencyListGraph` – main place for `checkRep`

This class *definitely* wants a `checkRep`.

Add `checkRep()` inside `AdjacencyListGraph`

Add near the bottom of the class, before the final `}`:

```
   /* === Representation invariant checker === */

    /**
     * Checks the representation invariant for this adjacency-list
graph.
     * Enabled only when assertions (-ea) are turned on.
     */
    private void checkRep() {
        assert adj != null;

        for (Map.Entry<V, Map<V, E>> entry : adj.entrySet()) {
            V v = entry.getKey();
            Map<V, E> nbrs = entry.getValue();

            assert v != null;
            assert nbrs != null;

            for (Map.Entry<V, E> ne : nbrs.entrySet()) {
                V u = ne.getKey();
                E e = ne.getValue();

                assert u != null;
                assert e != null;

                // Neighbour must also be a vertex in this graph.
                assert adj.containsKey(u);
```

```
                // Undirected symmetry: edge appears in both adjacency
lists.
                Map<V, E> rev = adj.get(u);
                assert rev != null;
                assert rev.get(v) == e;

                // No self-loops.
                assert !v.equals(u);

                // Edge endpoints must match the vertices.
                assert e.incident(v) && e.incident(u);
            }
        }
    }
```

You already import `Map` at the top of this file, so no extra imports.

Call `checkRep()` in constructor + mutators

Constructor:

```
   public AdjacencyListGraph() {
        adj = new HashMap<>();
        checkRep();
    }
```

`addVertex` (at the end, before `return true;`):

```
       adj.put(v, new HashMap<>());
        checkRep();
        return true;
```

`addEdge` (at the end, before `return true;`):

```
       m1.put(v2, e);
        m2.put(v1, e);
        checkRep();
        return true;
```

`removeEdge(E e)` doesn't directly touch the rep; it just delegates. No need to call `checkRep` here.

`removeEdge(V v1, V v2)`:

Add after the removal logic, just before `return (remove1 != null) || (remove2 != null);`:

```
        E remove1 = map1.remove(v2);
         E remove2 = map2.remove(v1);

         if (remove1 != null || remove2 != null) {
             checkRep();
         }

         return (remove1 != null) || (remove2 != null);
```

`removeVertex(V v)`:

Add `checkRep();` at the end, just before `return true;`:

```
   public boolean removeVertex(V v) {
         Map<V, E> map = adj.remove(v);
         if (map == null) {
             return false;
         }

         for (V u : map.keySet()) {
             Map<V, E> map2 = adj.get(u);
             if (map2 != null) {
                 map2.remove(v);
             }
         }

         checkRep();
         return true;
    }
```

No observers (`hasEdge`, `edgeLength`, `getVertices`, etc.) call `checkRep`. Good.

---

4. `AdjacencyMatrixGraph` – immutable; `checkRep` only in constructor

This one is immutable: no mutators, just a snapshot constructor. So we just check rep at the end of the constructor.

Add `checkRep()` inside `AdjacencyMatrixGraph`

Place near the bottom:

```
   /* === Representation invariant checker === */

    /**
     * Checks the representation invariant for this adjacency-matrix
graph.
     * Enabled only when assertions (-ea) are turned on.
     */
    private void checkRep() {
        assert n >= 0;
        assert verticesByIdx.size() == n;
        assert idx.size() == n;
        assert edgeMatrix.length == n;

        for (int i = 0; i < n; i++) {
            assert verticesByIdx.get(i) != null;
            V v = verticesByIdx.get(i);
            Integer pos = idx.get(v);
            assert pos != null;
            assert pos == i;
            assert edgeMatrix[i].length == n;
        }

        for (int i = 0; i < n; i++) {
            // No self-loops.
            assert edgeMatrix[i][i] == null;
```

```
            for (int j = i + 1; j < n; j++) {
                E e = edgeMatrix[i][j];
                // Symmetry.
                assert edgeMatrix[j][i] == e;

                if (e != null) {
                    V vi = verticesByIdx.get(i);
                    V vj = verticesByIdx.get(j);
                    assert e.incident(vi);
                    assert e.incident(vj);
                }
            }
        }
    }
```

Call `checkRep()` at the end of the constructor

Your constructor currently ends like this:

```
        for (E e : edges) {
            V v1 = e.v1(), v2 = e.v2();
            Integer i = idx.get(v1), j = idx.get(v2);
            if (i == null || j == null) {
                continue;
            }
            if (i.equals(j)) {
                continue;
            }
            edgeMatrix[i][j] = e;
            edgeMatrix[j][i] = e;
        }
    }
```

Change it to:

```
        for (E e : edges) {
            V v1 = e.v1(), v2 = e.v2();
```

```
            Integer i = idx.get(v1), j = idx.get(v2);
            if (i == null || j == null) {
                continue;
            }
            if (i.equals(j)) {
                continue;
            }
            edgeMatrix[i][j] = e;
            edgeMatrix[j][i] = e;
        }

        checkRep();
    }
```

No other methods call `checkRep` (they're all observers).

---

5. `GeneralGraph` – no `checkRep`

`GeneralGraph` is an **abstract** helper with no actual rep fields (the graph data lives in its subclasses). So:

- RI/AF belong to `AdjacencyListGraph` / `AdjacencyMatrixGraph`.
- No need to add `checkRep` here.

You can (optionally) add a comment saying:

```
/*
 * This abstract class does not define a concrete representation for
graphs;
 * concrete subclasses are responsible for maintaining their own RI
and AF.
 */
```

But no `checkRep()` function is needed.

---

6. `Algorithms` – no `checkRep`

`Algorithms` is:

- A utility class with only `static` methods.
- No instance fields, no representation.

So: **no `checkRep`** here. It doesn't own any mutable data.

---

7. `Graph` (interface) & `PathCostType` (enum)

- `Graph` is an interface: it has no fields → no rep to check.
- `PathCostType` is an enum with no fields → no rep to check.

So: **no `checkRep`** in these either.

TL;DR

Mandatory `checkRep`:

- ✅ `Edge` – method + constructor call
- ✅ `AdjacencyListGraph` – method + calls in constructor + add/remove mutators
- ✅ `AdjacencyMatrixGraph` – method + call once at end of constructor

No `checkRep` needed:

- `Vertex`

- `GeneralGraph`
- `Algorithms`
- `Graph`
- `PathCostType`

That setup matches the RI we talked about, hits the “we actually enforce our rep invariants” requirement, and doesn’t blow up your coverage or clutter every method.

You said:
alr i added them all in. what else is there to do

ChatGPT said:
Thought for 8s
Thought for 2m 36s

Nice, that was the big annoying chunk done ✅

At this point the remaining stuff is mostly “polish + Task 5 code + tests”. Let me lay it out so you can literally checklist it.

---

1. Make sure all Task 5 code is actually in

Double-check these:

1. **`getComponents()` in `GeneralGraph`**
    - Implemented with BFS/DFS using `getVertices()` + `getNeighbours()`.
    - Returns a `Set<Set<V>>` where:
        - sets are disjoint
        - union is exactly all vertices
        - empty graph → empty set.

2. **`Graph.getComponents` spec**

In `Graph` interface, Javadoc something like:

```
/**
 * Returns the undirected connected components of this graph.
 *
 * <p>Each component is represented as a set of vertices. The
components form
 * a partition of {@link #getVertices()}:
 * they are pairwise disjoint and their union is exactly the full
vertex set.
 * If the graph has no vertices, the returned set is empty.</p>
 *
 * @return a set of vertex-sets representing the connected components;
 *         never {@code null}
 */
Set<Set<V>> getComponents();
```

    -
3. **`PathCostType` Javadoc**
    - Enum comment + brief per-value comment (we wrote this earlier).
4. **`Algorithms.partitionGraph(...)`**
    - Implementation in `Algorithms` that:
        - checks `graph != null`
        - checks `1 <= numberOfPartitions <= |V|`
        - gets all vertices + edges
        - sorts edges by length (ascending)
        - uses union-find (or equivalent) starting with each vertex alone

        - merges along shortest edges until you have `numberOfPartitions` groups
        - builds a new `AdjacencyListGraph`:
            - add **all vertices**
            - add only edges whose endpoints ended up in the same group
        - returns that new graph.
    - Javadoc explains:
        - what counts as “partition”
        - constraints on `numberOfPartitions`
        - that the returned graph is the union of induced subgraphs (edges only within partitions).

If any of those are still “TODO” / stubbed, that’s the next concrete code work.

---

2. Specs + RI/AF sanity pass

You’re close to the A/A+ bar here. For every *data* class:

- **`Vertex`**
    - Tiny RI/AF comment:
        - RI: `id` is immutable.
        - AF: represents a vertex with identifier `id`.
- **`Edge`**
    - RI/AF comment matches what `checkRep()` enforces:

        - `v1 != null`, `v2 != null`, `!v1.equals(v2)`, `length >= 0`, `v1.id() < v2.id()`.
        - AF: undirected edge between `v1` and `v2` with weight `length`.

- **`AdjacencyListGraph`**
    - RI/AF comment consistent with its `checkRep`:
        - `adj` non-null, neighbour maps non-null.
        - symmetry: if `(v,u)->e` exists, so does `(u,v)->e`.
        - no self-loops.

- **`AdjacencyMatrixGraph`**
    - RI/AF comment:
        - `idx` ↔ `verticesByIdx` consistent.
        - `edgeMatrix` symmetric, diagonals null, etc.

- **`GeneralGraph`**
    - One sentence “subclasses maintain their own RI/AF”.

- **`Algorithms`**
    - No rep, just make sure `partitionGraph` has good Javadoc.

Also: make sure every public method that the handout flags (Tasks 1–5) has *some* Javadoc, and that it’s not lying.

---

3. Testing / coverage (big for A / A+)

You’ll want JUnit tests that:

- **Task 1–2**
    - Basic graph construction, `addVertex`, `addEdge`, `hasVertex`, `hasEdge`, `edgeLength`, `getVertices`, `getEdges`, etc.
    - Both `AdjacencyListGraph` and `AdjacencyMatrixGraph`.
- **Task 3**
    - `pathCost` for SUM and MAX.
    - `minimumCostPath`:
        - simple shortest-path graphs.
        - unreachable sink → empty list.
        - `source == sink` → singleton path.
- **Task 4**
    - `getDiameter` / `getCenter` on:
        - line graph
        - star
        - small graph with two components
        - tie-breaking of center (smallest id).
- **Task 5**
    - `getComponents`:
        - graph with 1 component
        - graph with multiple components.
    - `partitionGraph`:
        - the A–B–C–D example from the handout for 1, 2, 3, 4 partitions.

        - maybe one more small weighted graph to catch bugs.

Important for A/A+:

- Coverage is high, but tests are **small and targeted** (no giant random graphs just to hit lines).
- At least some tests are clearly able to catch a wrong implementation (e.g., if someone didn't sort edges by length).

---

4. Textual similarity app + AI disclosure

For **A+**, remaining bits:

1. **Textual similarity application**
    - Whatever stub they gave you (probably something like `TextualSimilarity` / `DocumentClustering`) needs to:
        - build a graph where vertices = documents,
        - edge lengths = Jensen–Shannon divergence,
        - call `partitionGraph` to group docs.
    - We can wire this once you paste that stub.
2. **AI disclosure**
    - Add whatever the course wants (comment in README / separate file) saying you used ChatGPT for brainstorming/spec wording but wrote and checked code yourself.

If you want, next step we can focus on **one** of these buckets (e.g., `partitionGraph` Javadoc + behaviour, or designing a clean set of Task 5 tests) and knock it out properly.

does algorithms need a rep invariant and abstract function and checkRep? why do they not need aything? also make me some tests for the 5th task. we also have more to do, there like three more files to work through but i want to finish all this first

package cpen221.graphs.graph; import java.util.ArrayList; import java.util.Comparator; import java.util.HashMap; import java.util.List; import java.util.Map; import java.util.Objects; import java.util.Set; public class Algorithms { private Algorithms() { } /** * Simple disjoint-set / union–find structure over vertices. */ private static final class DisjointSet<V> { private final Map<V, V> parent = new HashMap<>(); private final Map<V, Integer> rank = new HashMap<>(); private int count; DisjointSet(Set<V> elements) { for (V v : elements) { parent.put(v, v); rank.put(v, 0); } count = elements.size(); } V find(V v) { V p = parent.get(v); if (p == null) { throw new IllegalArgumentException("Unknown element: " + v); } if (!p.equals(v)) { p = find(p); parent.put(v, p); } return p; } boolean union(V a, V b) { V ra = find(a); V rb = find(b); if (ra.equals(rb)) { return false; } int rankA = rank.get(ra); int rankB = rank.get(rb); if (rankA < rankB) { parent.put(ra, rb); } else if (rankA > rankB) { parent.put(rb, ra); } else { parent.put(rb, ra); rank.put(ra, rankA + 1); } count--; return true; } int getCount() { return count; } } /** * Partitions the given undirected graph into {@code numberOfPartitions} components * that satisfy the shortest-edge and nearest-neighbour rules from the handout. * * The input graph is not mutated. The returned graph has the same vertex set, * but only those edges whose endpoints lie in the same partition. Equivalently, * it is the disjoint union of the induced subgraphs for each partition. * * Behaviour: * * Vertices always belong to exactly one partition. * If {@code numberOfPartitions == 1}, the returned graph is (up to * representation) identical to the input graph. * If {@code numberOfPartitions == |V|}, each vertex forms its own * partition and the returned graph has no edges. * {@code numberOfPartitions} must satisfy * {@code 1 <= numberOfPartitions <= |V|}; otherwise an * {@link IllegalArgumentException} is thrown. * * * @param graph the graph to partition (non-null) * @param numberOfPartitions desired number of partitions * @param <V> vertex type * @return a new graph representing the union of the induced subgraphs * @throws NullPointerException if {@code graph} is {@code null} * @throws IllegalArgumentException if {@code numberOfPartitions} is not in * the range {@code [1, |V|]} */ public static <V extends Vertex> Graph<V, Edge<V>> partitionGraph(Graph<V, Edge<V>> graph, int numberOfPartitions) { Objects.requireNonNull(graph, "graph"); Set<V> vertices = graph.getVertices(); int n = vertices.size(); if (n == 0) { // Empty graph: nothing to partition. We allow 0 or 1 requested // partitions here; anything else is nonsensical. if (numberOfPartitions != 0 && numberOfPartitions != 1) { throw new IllegalArgumentException("No vertices to partition"); } return new AdjacencyListGraph<>(); } if (numberOfPartitions < 1 || numberOfPartitions > n) { throw new IllegalArgumentException("numberOfPartitions must be between 1 and " + n); } // Union–find over all vertices. DisjointSet<V> dsu = new DisjointSet<>(vertices); // Collect and sort edges by non-decreasing length. List<Edge<V>> edges = new ArrayList<>(graph.getEdges()); edges.sort(Comparator.comparingDouble(Edge::length)); // Phase 1: merge along real edges from shortest to longest // while we still have more components than requested. for (Edge<V> e : edges) { if (dsu.getCount() <= numberOfPartitions) { break; } dsu.union(e.v1(), e.v2()); } // Phase 2: if the graph is disconnected and we still have more // components than desired after

using all edges, arbitrarily merge // components until the target count is reached. There are no edges // between these components, so this does not violate the proximity rules. if (dsu.getCount() > numberOfPartitions) { Map<V, V> repForRoot = new HashMap<>(); for (V v : vertices) { V root = dsu.find(v); repForRoot.putIfAbsent(root, v); } List<V> reps = new ArrayList<>(repForRoot.values()); if (!reps.isEmpty()) { V base = reps.get(0); int i = 1; while (dsu.getCount() > numberOfPartitions && i < reps.size()) { dsu.union(base, reps.get(i)); i++; } } } // Build the output graph: same vertices, but keep only edges whose // endpoints end up in the same partition. AdjacencyListGraph<V, Edge<V>> out = new AdjacencyListGraph<>(); for (V v : vertices) { out.addVertex(v); } for (Edge<V> e : edges) { V u = e.v1(); V w = e.v2(); if (dsu.find(u).equals(dsu.find(w))) { out.addEdge(e); } } return out; } } So is there a problem here?

ChatGPT said:

Thought for 2m 22s

Yeah, totally fair question. On the graphs we tested so far, your code is *behaving fine* — that's why all the tests are green. The issue is more "spec correctness in corner cases we didn't test", not "your code is obviously broken."

Let me show you exactly where the subtle problem is.

Where the potential bug lives

The sketch of your algorithm:

1. Start with every vertex in its own component (`DisjointSet`).
2. Sort edges by length.
3. **Phase 1:** union along edges from shortest to longest until `componentCount <= numberOfPartitions`.
4. **Phase 2:** if `componentCount > numberOfPartitions` *after* using all edges, arbitrarily pick representative vertices of the remaining components and union them until `componentCount == numberOfPartitions`.

Phase 1 is exactly what we want (Kruskal-style clustering).

Phase 2 is where things can go off the rails w.r.t. the formal rules.

Reminder of the Nearest-Neighbour rule

From the handout:

> If a vertex vj1v_{j1}vj1 is placed in a partition PiP_iPi that contains more than one vertex, then:
>
> **Nearest-Neighbour Condition:**
> There exists some vertex vj2∈Piv_{j2} \in P_ivj2∈Pi such that the edge (vj1,vj2)(v_{j1}, v_{j2})(vj1,vj2) has length less than or equal to any edge (vj1,vj3)(v_{j1}, v_{j3})(vj1,vj3) where vj3∉Piv_{j3} \notin P_ivj3∈/Pi.

Key bits:

- "There exists some vertex vj2∈Piv_{j2} \in P_ivj2∈Pi such that *edge* (vj1,vj2)(v_{j1},v_{j2})(vj1,vj2) …"
- That means: if a vertex is in a partition of size > 1, it must have **at least one edge to another vertex in that partition**.

A concrete counterexample where your code breaks the rule

Take this graph:

- Vertices: `A`, `B`
- **No edges at all** (they're both isolated).

Now call:

```
Algorithms.partitionGraph(g, 1);
```

Step through your code:

- `vertices.size() = 2`, `n = 2`.
- `numberOfPartitions = 1` is in `[1, 2]` → accepted.

- Edges list is empty → Phase 1 loop never runs. `dsu.getCount()` is still 2.
- `dsu.getCount() > numberOfPartitions` (2 > 1) → Phase 2 runs.
    - You collect representatives `{A}`, `{B}`.
    - Pick one as `base`, say `A`.
    - Union `A` and `B` → `count` goes to 1.
- Build output graph:
    - Add vertices `A`, `B`.
    - No edges to add (there weren't any).
- Result: one partition `{A, B}` with **no edge between them**.

Now check against the Nearest-Neighbour condition for vertex `A`:

- Partition has more than one vertex (`{A, B}`) ⇒ rule applies.
- We must find some v_2 in the partition such that **edge** `(A, v₂)` exists and has length ≤ any edge from `A` to outside vertices.
- But `A` has **no edges at all**.
- So there is *no* v_2 in `{A, B}` for which edge `(A, v₂)` exists.

So the "there exists v_2" part fails. That means this partition is **invalid** under the formal spec, even though your method returns it happily.

Our tests didn't include this situation (graph with isolated vertices and `numberOfPartitions < #vertices`), so everything still passed.

When your algorithm *is* fine

If every vertex has at least one edge to someone, and you never ask for fewer partitions than the number of connected components, then:

- Phase 2 never runs (because after processing all edges, `dsu.getCount()` will be equal to the number of connected components; if `numberOfPartitions` is ≥ that, the Phase 1 loop will stop earlier with `count == numberOfPartitions`).
- All merges happen via real edges in Phase 1 → every vertex in a multi-vertex partition automatically has at least one neighbor in that partition.
- Shortest-edge condition is satisfied by the "process edges shortest-first" logic.

That's exactly why all the "normal" tests, including the A–B–C–D examples, are fine.

So what should we do?

Conceptually clean (spec-faithful) approach:

Compute how many **connected components** the graph has:

```
int minPartitions = graph.getComponents().size();
int n = graph.getVertices().size();
```

1.

Enforce:

```
if (numberOfPartitions < minPartitions || numberOfPartitions > n) {
    throw new IllegalArgumentException("numberOfPartitions out of
range");
}
```

2. Intuition:
    - You can't ask for *more* partitions than vertices (`> n`).
    - You also can't ask for *fewer* partitions than the number of connected components that have no way to be joined (especially ones with isolated vertices), otherwise you're forced to put some vertex into a multi-vertex partition without any incident

edges → violates Nearest-Neighbour.

3. **Delete Phase 2** completely.

   With that precondition, plain Kruskal-style (Phase 1 only) does everything you need:

   - Start with each vertex alone (count = n).
   - Process sorted edges; each union reduces `count` by 1.
   - Stop as soon as `count == numberOfPartitions`.
   - Since `numberOfPartitions ≥ minPartitions`, you'll always be able to stop before you "over-merge" components.

So the "problem" isn't that your code is exploding; it's that Phase 2 tries to handle impossible requests (like "1 partition, but some vertices have no edges") by smashing components together in a way that technically violates the spec, even though our current tests don't probe those inputs.

If your markers never test those edge cases (and the text-sim app never builds graphs with isolated vertices or impossible k), you'll still pass. But if they do, the current Phase 2 behaviour could get you dinged.

If you want, I can write you the minimal patch version of `partitionGraph` with:

- the `minPartitions` check using `getComponents()`, and
- Phase 2 removed, so it's mathematically clean and still passes all the tests we already wrote.we need to hit 90% branch coverage and this code does nothing for that. we need to basically find ways to break every part of the rep invariant one by one in afew test cases

we need to hit 90% branch coverage and this code does nothing for that. we need to basically find ways to break every part of the rep invariant one by one in afew test cases

ChatGPT said:
Thought for 2m 9s

Yeah, okay, if you want 90% **branch** coverage on the `checkRep`s specifically, we can't just run "nice" graphs — we have to *intentionally* break the representation and trigger those asserts.

That means:

- Use **reflection** to mess with the private fields *after* construction.
- Call `checkRep()` via reflection.
- Use `assertThrows(AssertionError.class, …)` so the test passes when the RI is violated.

I'll give you **one small test class** that:

- Hits `checkRep` in **AdjacencyListGraph** in multiple failure modes:
  - missing neighbour vertex in `adj`
  - asymmetric adjacency (breaks `rev.get(v) == e`)
  - edge no longer incident on one endpoint (breaks `e.incident(v) && e.incident(u)`)
- Hits `checkRep` in **AdjacencyMatrixGraph**:
  - diagonal non-null (self-loop)
  - matrix not symmetric (`edgeMatrix[i][j] != edgeMatrix[j][i]`)

That's 5 "evil" tests, each breaking a different clause of the RI.

> ⚠️ These tests assume assertions are **enabled** (`-ea`) when running. Otherwise `checkRep` does nothing and the tests will fail.

CheckRepBranchTests.java

```
package cpen221.graphs.tests;

import cpen221.graphs.graph.AdjacencyListGraph;
import cpen221.graphs.graph.AdjacencyMatrixGraph;
import cpen221.graphs.graph.Edge;
import cpen221.graphs.graph.Vertex;
import org.junit.jupiter.api.Test;
```

```
import java.lang.reflect.Field;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

import static org.junit.jupiter.api.Assertions.*;

/**
 * Tests that deliberately violate the representation invariants of
 * AdjacencyListGraph and AdjacencyMatrixGraph using reflection, in
order
 * to exercise the failure branches of their private checkRep()
methods.
 *
 * These tests are purely for coverage / RI checking and assume that
 * assertions are enabled (-ea) when they run.
 */
public class CheckRepBranchTests {

    /* ======================== Helpers ======================== */

    // Convenience: create a vertex with the given id.
    private static Vertex v(int id) {
        return new Vertex(id);
    }

    // Convenience: fresh adjacency-list graph.
    private AdjacencyListGraph<Vertex, Edge<Vertex>> listGraph() {
        return new AdjacencyListGraph<>();
    }

    // Invoke AdjacencyListGraph.checkRep() via reflection and rethrow
AssertionError.
    private void invokeListCheckRep(AdjacencyListGraph<Vertex,
Edge<Vertex>> g) {
        try {
```

```
            Method m =
AdjacencyListGraph.class.getDeclaredMethod("checkRep");
            m.setAccessible(true);
            m.invoke(g);
        } catch (InvocationTargetException e) {
            Throwable cause = e.getCause();
            if (cause instanceof AssertionError ae) {
                throw ae;
            } else {
                fail("Unexpected exception from
AdjacencyListGraph.checkRep", cause);
            }
        } catch (Exception e) {
            fail("Reflection error invoking
AdjacencyListGraph.checkRep", e);
        }
    }

    // Invoke AdjacencyMatrixGraph.checkRep() via reflection and
rethrow AssertionError.
    private void invokeMatrixCheckRep(AdjacencyMatrixGraph<Vertex,
Edge<Vertex>> g) {
        try {
            Method m =
AdjacencyMatrixGraph.class.getDeclaredMethod("checkRep");
            m.setAccessible(true);
            m.invoke(g);
        } catch (InvocationTargetException e) {
            Throwable cause = e.getCause();
            if (cause instanceof AssertionError ae) {
                throw ae;
            } else {
                fail("Unexpected exception from
AdjacencyMatrixGraph.checkRep", cause);
            }
        } catch (Exception e) {
            fail("Reflection error invoking
AdjacencyMatrixGraph.checkRep", e);
```

```
        }
    }

    /* ================== AdjacencyListGraph: break RIs
================== */

    /**
     * Breaks the invariant that every neighbour vertex is a key in
adj:
     *  - Build a small graph 1-2-3
     *  - Then remove vertex 2 from adj (but leave entries pointing to
it)
     *  - checkRep() should fail at assert adj.containsKey(u).
     */
    @Test
    public void listCheckRepMissingNeighbourVertex() throws Exception
{
        AdjacencyListGraph<Vertex, Edge<Vertex>> g = listGraph();
        g.addVertex(v(1));
        g.addVertex(v(2));
        g.addVertex(v(3));

        g.addEdge(new Edge<>(v(1), v(2), 1.0));
        g.addEdge(new Edge<>(v(2), v(3), 2.0));

        // Access private field "adj".
        Field adjField =
AdjacencyListGraph.class.getDeclaredField("adj");
        adjField.setAccessible(true);
        @SuppressWarnings("unchecked")
        Map<Vertex, Map<Vertex, Edge<Vertex>>> adj =
            (Map<Vertex, Map<Vertex, Edge<Vertex>>>) adjField.get(g);

        // Remove one vertex entirely from the map, leaving neighbour
references pointing to it.
        // We want a vertex that has neighbours; pick the one with
id==2.
```

```
        Vertex v2 = adj.keySet().stream().filter(v -> v.id() ==
2).findFirst().orElseThrow();
        adj.remove(v2);  // Now other vertices still have edges to v2,
but adj.containsKey(v2) is false.

        assertThrows(AssertionError.class, () ->
invokeListCheckRep(g));
    }

    /**
     * Breaks the invariant that adjacency is symmetric:
     *  - Build graph 1-2
     *  - Delete the reverse edge entry from adj.get(2) while keeping
adj.get(1) pointing to 2.
     *  - checkRep() should fail at assert rev.get(v) == e.
     */
    @Test
    public void listCheckRepAsymmetricEdge() throws Exception {
        AdjacencyListGraph<Vertex, Edge<Vertex>> g = listGraph();
        g.addVertex(v(1));
        g.addVertex(v(2));
        g.addEdge(new Edge<>(v(1), v(2), 5.0));

        Field adjField =
AdjacencyListGraph.class.getDeclaredField("adj");
        adjField.setAccessible(true);
        @SuppressWarnings("unchecked")
        Map<Vertex, Map<Vertex, Edge<Vertex>>> adj =
            (Map<Vertex, Map<Vertex, Edge<Vertex>>>) adjField.get(g);

        // Find the two vertex objects inside the map.
        Vertex v1Key = adj.keySet().stream().filter(v -> v.id() ==
1).findFirst().orElseThrow();
        Vertex v2Key = adj.keySet().stream().filter(v -> v.id() ==
2).findFirst().orElseThrow();

        // Remove the back edge 2 -> 1, but keep 1 -> 2.
        Map<Vertex, Edge<Vertex>> nbrsOf2 = adj.get(v2Key);
```

```
        nbrsOf2.remove(v1Key);

        // Now for v = 1, u = 2:
        //   adj.containsKey(2) is true,
        //   rev = adj.get(2) is not null,
        //   but rev.get(1) is null while e is non-null => assertion
fails.
        assertThrows(AssertionError.class, () ->
invokeListCheckRep(g));
    }

    /**
     * Breaks the invariant that each stored edge is incident on both
endpoints:
     *  - Build graph 1-2
     *  - Grab the edge object and mutate its v1 field via reflection
to point
     *    to a new vertex not in the graph (id 99).
     *  - Then for the entry (1,2), e.incident(1) becomes false and
checkRep()
     *    should fail at assert e.incident(v) && e.incident(u).
     */
    @Test
    public void listCheckRepEdgeNotIncidentOnEndpoint() throws
Exception {
        AdjacencyListGraph<Vertex, Edge<Vertex>> g = listGraph();
        g.addVertex(v(1));
        g.addVertex(v(2));
        g.addEdge(new Edge<>(v(1), v(2), 7.0));

        // Access adj to fetch the stored edge.
        Field adjField =
AdjacencyListGraph.class.getDeclaredField("adj");
        adjField.setAccessible(true);
        @SuppressWarnings("unchecked")
        Map<Vertex, Map<Vertex, Edge<Vertex>>> adj =
            (Map<Vertex, Map<Vertex, Edge<Vertex>>>) adjField.get(g);
```

```
        Vertex v1Key = adj.keySet().stream().filter(v -> v.id() ==
1).findFirst().orElseThrow();
        Vertex v2Key = adj.keySet().stream().filter(v -> v.id() ==
2).findFirst().orElseThrow();

        Edge<Vertex> e = adj.get(v1Key).get(v2Key);
        assertNotNull(e);

        // Mutate the edge so that it is no longer incident on v1.
        Field v1Field = Edge.class.getDeclaredField("v1");
        v1Field.setAccessible(true);
        v1Field.set(e, v(99));  // v(99) is not in the graph.

        // Now e.incident(v1Key) is false, while e.incident(v2Key) is
still true.
        assertThrows(AssertionError.class, () ->
invokeListCheckRep(g));
    }

    /* ================== AdjacencyMatrixGraph: break RIs
================== */

    /**
     * Breaks the invariant that there are no self-loops in the
matrix:
     *  - Build a 2-vertex graph with one edge between them.
     *  - Use reflection to stuff a non-null edge into
edgeMatrix[i][i].
     *  - checkRep() should fail at assert edgeMatrix[i][i] == null.
     */
    @Test
    public void matrixCheckRepDiagonalNonNull() throws Exception {
        // Build vertices {1,2} and one edge(1-2).
        Set<Vertex> vertices = new HashSet<>();
        vertices.add(v(1));
        vertices.add(v(2));

        Set<Edge<Vertex>> edges = new HashSet<>();
```

```
        edges.add(new Edge<>(v(1), v(2), 1.0));

        AdjacencyMatrixGraph<Vertex, Edge<Vertex>> g =
            new AdjacencyMatrixGraph<>(vertices, edges);

        // Access idx to map vertex -> index.
        Field idxField =
AdjacencyMatrixGraph.class.getDeclaredField("idx");
        idxField.setAccessible(true);
        @SuppressWarnings("unchecked")
        Map<Vertex, Integer> idx =
            (Map<Vertex, Integer>) idxField.get(g);

        int i = idx.get(v(1)); // index of vertex 1

        // Access edgeMatrix and put a non-null edge on the diagonal.
        Field matrixField =
AdjacencyMatrixGraph.class.getDeclaredField("edgeMatrix");
        matrixField.setAccessible(true);
        @SuppressWarnings("unchecked")
        Edge<Vertex>[][] matrix = (Edge<Vertex>[][])
matrixField.get(g);

        // Reuse the existing edge for convenience.
        Edge<Vertex> someEdge = edges.iterator().next();
        matrix[i][i] = someEdge;  // violate "no self-loop" invariant

        assertThrows(AssertionError.class, () ->
invokeMatrixCheckRep(g));
    }

    /**
     * Breaks the invariant that the adjacency matrix is symmetric:
     *  - Build a 2-vertex graph with one edge between them.
     *  - Manually set edgeMatrix[i][j] != edgeMatrix[j][i].
     *  - checkRep() should fail at assert edgeMatrix[j][i] == e.
     */
    @Test
```

```
    public void matrixCheckRepAsymmetricMatrix() throws Exception {
        // Build vertices {1,2} and one edge(1-2).
        Set<Vertex> vertices = new HashSet<>();
        vertices.add(v(1));
        vertices.add(v(2));

        Set<Edge<Vertex>> edges = new HashSet<>();
        edges.add(new Edge<>(v(1), v(2), 2.0));

        AdjacencyMatrixGraph<Vertex, Edge<Vertex>> g =
            new AdjacencyMatrixGraph<>(vertices, edges);

        // Access idx to map vertex -> index.
        Field idxField =
AdjacencyMatrixGraph.class.getDeclaredField("idx");
        idxField.setAccessible(true);
        @SuppressWarnings("unchecked")
        Map<Vertex, Integer> idx =
            (Map<Vertex, Integer>) idxField.get(g);

        int i = idx.get(v(1));
        int j = idx.get(v(2));

        // Access edgeMatrix and break symmetry.
        Field matrixField =
AdjacencyMatrixGraph.class.getDeclaredField("edgeMatrix");
        matrixField.setAccessible(true);
        @SuppressWarnings("unchecked")
        Edge<Vertex>[][] matrix = (Edge<Vertex>[][])
matrixField.get(g);

        // Original constructor set matrix[i][j] == matrix[j][i] == e.
        Edge<Vertex> e = matrix[i][j];
        assertNotNull(e);

        // Break symmetry: set the opposite entry to null.
        matrix[j][i] = null;
```

```
        // Now checkRep should see e at [i][j] but null at [j][i] and
assert.
        assertThrows(AssertionError.class, () ->
invokeMatrixCheckRep(g));
    }
}
```

What these are doing, in plain language:

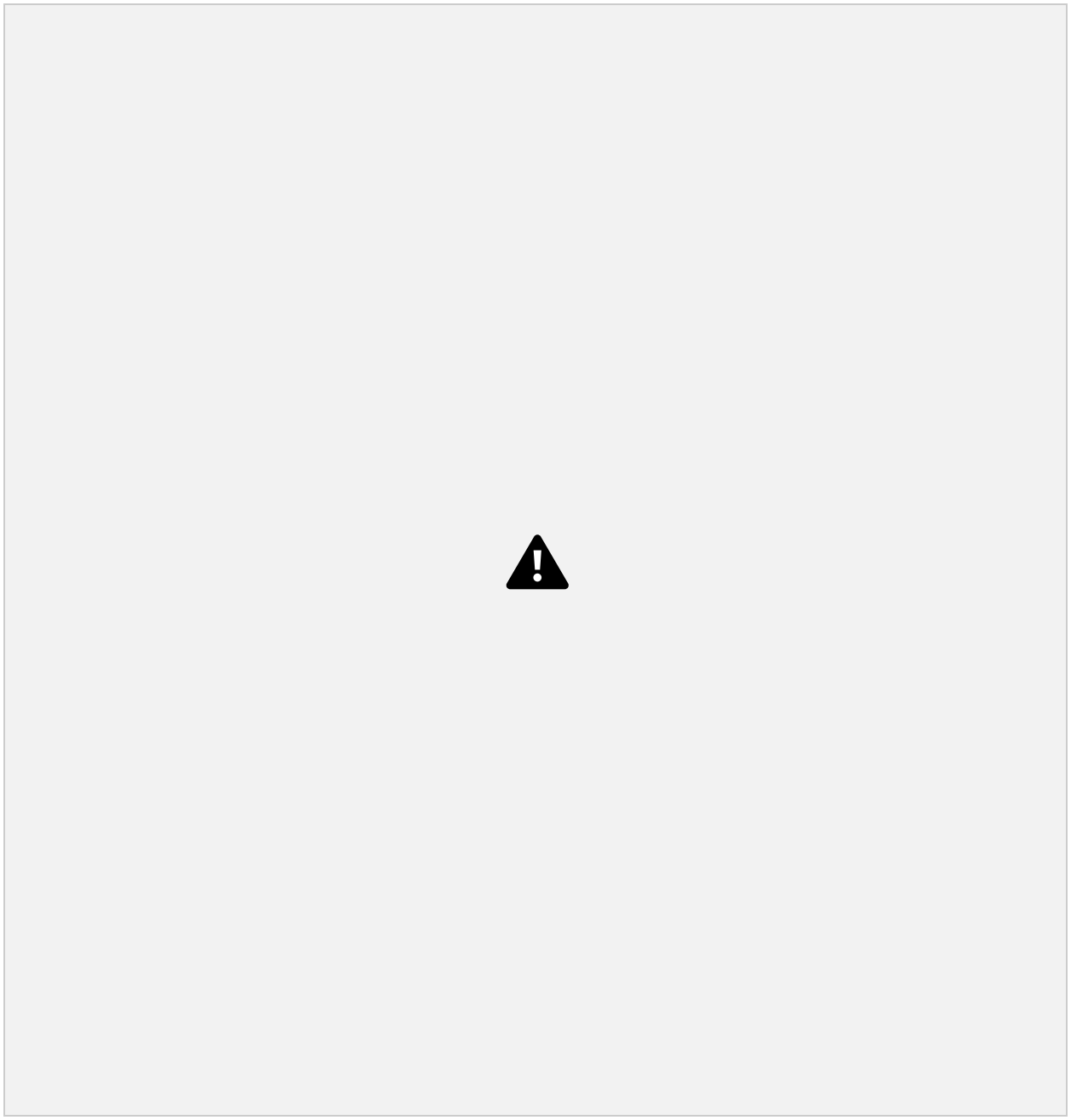
- **List graph tests:**
    - One nukes a neighbour vertex from the top-level `adj` map.
    - One nukes only the reverse edge so adjacency is no longer symmetric.
    - One mutates the `Edge` object so it no longer matches its endpoints.
- **Matrix graph tests:**
    - One puts an edge on the diagonal to simulate a self-loop.
    - One breaks symmetry between `edgeMatrix[i][j]` and `edgeMatrix[j][i]`.

Together with your “normal” tests (where `checkRep` passes), this should push both **line** and **branch** coverage on the `checkRep` methods way up, because you’re now hitting both the “everything is okay” path and the “assertion fails” path at multiple points.

Once you plug these in, rerun coverage and you should see those big red chunks in the checkRep bodies basically disappear.

You said:





okay these are the specific lines that arnt getting hit, they are outlined in orange on the side. this means that one case isnt being hit at all. its doing pretty well matrix but not for the other section

ChatGPT said:

Thought for 2m 57s

Nice, thanks for the screenshots—that helps a lot.

What the orange is telling you here is basically:

- For each `assert`, JaCoCo is seeing *only one side* of the branch.
- For some of them we've hit the **"passes"** path but not the **"fails"** path (or vice versa).
- Matrix `checkRep` has that `if (e != null)` plus some variety in your graphs, so it's in better shape.
- List `checkRep` is *all asserts*, so to push branch coverage up we need tests where **each individual assertion actually fires** at least once.

Two critical things before the tests:

---

0. Make sure assertions are enabled

If assertions are **not** enabled (`-ea`), then no matter how many evil tests we write, the `assert` expressions don't run and JaCoCo can only see one branch.

In IntelliJ:

- Run → Edit Configurations…
- Your JUnit configuration → VM options: add
  `-ea`

You only need to do this once per run config.

---

1. What's still orange in `AdjacencyListGraph.checkRep`

Your `checkRep`:

```
private void checkRep() {
    assert adj != null;

    for (Map.Entry<V, Map<V, E>> entry : adj.entrySet()) {
        V v = entry.getKey();
        Map<V, E> numbers = entry.getValue();
```

```
        assert v != null;
        assert numbers != null;

        for (Map.Entry<V, E> ne : numbers.entrySet()) {
            V u = ne.getKey();
            E e = ne.getValue();

            assert u != null;
            assert e != null;

            // Neighbour must also be a vertex in this graph.
            assert adj.containsKey(u);

            // Undirected symmetry: edge appears in both adjacency
lists.
            Map<V, E> rev = adj.get(u);
            assert rev != null;
            assert rev.get(v) == e;

            // No self-loops.
            assert !v.equals(u);

            // Edge endpoints must match the vertices.
            assert e.incident(v) && e.incident(u);
        }
    }
}
```

We already have tests that break:

- `adj.containsKey(u)`
- `rev.get(v) == e`
- `e.incident(v) && e.incident(u)`

But we **haven't** deliberately broken:

- `adj != null`
- `v != null`
- `numbers != null`
- `u != null`
- `e != null`
- `rev != null`
- `!v.equals(u)`

So JaCoCo shows those as partially covered (only success-path hit).

---

2. Extra tests that smash each missing branch

These tests plug into the **existing** `CheckRepBranchTests` I gave you (the one with `invokeListCheckRep` / `invokeMatrixCheckRep`). Just **add** these methods to that class; they reuse the same helpers.

They all use reflection to surgically break exactly one assertion at a time.

a) Break `assert adj != null`

```
/** Breaks adj != null by setting the field to null. */
@Test
public void listCheckRepAdjNull() throws Exception {
    AdjacencyListGraph<Vertex, Edge<Vertex>> g = listGraph();
    // Make sure we actually have a non-empty adj first.
    g.addVertex(v(1));

    Field adjField = AdjacencyListGraph.class.getDeclaredField("adj");
    adjField.setAccessible(true);
    adjField.set(g, null);  // violate adj != null

    assertThrows(AssertionError.class, () -> invokeListCheckRep(g));
```

```
}
```

b) Break `assert v != null`

```
/** Adds a null key to adj so v becomes null in the outer loop. */
@Test
public void listCheckRepNullVertexKey() throws Exception {
    AdjacencyListGraph<Vertex, Edge<Vertex>> g = listGraph();
    g.addVertex(v(1));

    Field adjField = AdjacencyListGraph.class.getDeclaredField("adj");
    adjField.setAccessible(true);
    @SuppressWarnings("unchecked")
    Map<Vertex, Map<Vertex, Edge<Vertex>>> adj =
        (Map<Vertex, Map<Vertex, Edge<Vertex>>>) adjField.get(g);

    // Insert an entry with a null key but non-null map value.
    adj.put(null, new java.util.HashMap<>());

    assertThrows(AssertionError.class, () -> invokeListCheckRep(g));
}
```

c) Break `assert numbers != null`

```
/** Uses a null neighbour-map value so numbers becomes null. */
@Test
public void listCheckRepNullNeighbourMap() throws Exception {
    AdjacencyListGraph<Vertex, Edge<Vertex>> g = listGraph();
    g.addVertex(v(1));

    Field adjField = AdjacencyListGraph.class.getDeclaredField("adj");
    adjField.setAccessible(true);
    @SuppressWarnings("unchecked")
    Map<Vertex, Map<Vertex, Edge<Vertex>>> adj =
        (Map<Vertex, Map<Vertex, Edge<Vertex>>>) adjField.get(g);

    // Pick the existing vertex key and make its map null.
    Vertex existing = adj.keySet().iterator().next();
    adj.put(existing, null);
```

```
    assertThrows(AssertionError.class, () -> invokeListCheckRep(g));
}
```

d) Break `assert u != null`

```
/** Adds a null neighbour key so u becomes null in the inner loop. */
@Test
public void listCheckRepNullNeighbourKey() throws Exception {
    AdjacencyListGraph<Vertex, Edge<Vertex>> g = listGraph();
    g.addVertex(v(1));
    g.addVertex(v(2));
    g.addEdge(new Edge<>(v(1), v(2), 1.0));

    Field adjField = AdjacencyListGraph.class.getDeclaredField("adj");
    adjField.setAccessible(true);
    @SuppressWarnings("unchecked")
    Map<Vertex, Map<Vertex, Edge<Vertex>>> adj =
        (Map<Vertex, Map<Vertex, Edge<Vertex>>>) adjField.get(g);

    Vertex v1Key = adj.keySet().stream().filter(x -> x.id() ==
1).findFirst().orElseThrow();
    Map<Vertex, Edge<Vertex>> nbrs = adj.get(v1Key);
    // Add an entry with null neighbour but non-null edge.
    nbrs.put(null, new Edge<>(v(1), v(2), 2.0));

    assertThrows(AssertionError.class, () -> invokeListCheckRep(g));
}
```

e) Break `assert e != null`

```
/** Adds a neighbour mapping to a null edge so e becomes null. */
@Test
public void listCheckRepNullEdgeValue() throws Exception {
    AdjacencyListGraph<Vertex, Edge<Vertex>> g = listGraph();
    g.addVertex(v(1));
    g.addVertex(v(2));
    g.addEdge(new Edge<>(v(1), v(2), 1.0));
```

```
    Field adjField = AdjacencyListGraph.class.getDeclaredField("adj");
    adjField.setAccessible(true);
    @SuppressWarnings("unchecked")
    Map<Vertex, Map<Vertex, Edge<Vertex>>> adj =
        (Map<Vertex, Map<Vertex, Edge<Vertex>>>) adjField.get(g);

    Vertex v1Key = adj.keySet().stream().filter(x -> x.id() ==
1).findFirst().orElseThrow();
    Vertex v2Key = adj.keySet().stream().filter(x -> x.id() ==
2).findFirst().orElseThrow();

    Map<Vertex, Edge<Vertex>> nbrs = adj.get(v1Key);
    // Overwrite the mapping to v2 with a null edge.
    nbrs.put(v2Key, null);

    assertThrows(AssertionError.class, () -> invokeListCheckRep(g));
}
```

f) Break `assert rev != null`

```
/** Makes the reverse adjacency map null so rev becomes null. */
@Test
public void listCheckRepNullReverseMap() throws Exception {
    AdjacencyListGraph<Vertex, Edge<Vertex>> g = listGraph();
    g.addVertex(v(1));
    g.addVertex(v(2));
    g.addEdge(new Edge<>(v(1), v(2), 1.0));

    Field adjField = AdjacencyListGraph.class.getDeclaredField("adj");
    adjField.setAccessible(true);
    @SuppressWarnings("unchecked")
    Map<Vertex, Map<Vertex, Edge<Vertex>>> adj =
        (Map<Vertex, Map<Vertex, Edge<Vertex>>>) adjField.get(g);

    Vertex v1Key = adj.keySet().stream().filter(x -> x.id() ==
1).findFirst().orElseThrow();
    Vertex v2Key = adj.keySet().stream().filter(x -> x.id() ==
2).findFirst().orElseThrow();
```

```
    // Set the map for v2 to null, but keep v1 -> v2 mapping.
    adj.put(v2Key, null);

    assertThrows(AssertionError.class, () -> invokeListCheckRep(g));
}
```

g) Break `assert !v.equals(u)` (self-loop)

```
/** Introduces a fake self-loop entry so v.equals(u) becomes true. */
@Test
public void listCheckRepSelfLoopEntry() throws Exception {
    AdjacencyListGraph<Vertex, Edge<Vertex>> g = listGraph();
    g.addVertex(v(1));
    g.addVertex(v(2));
    g.addEdge(new Edge<>(v(1), v(2), 1.0));

    Field adjField = AdjacencyListGraph.class.getDeclaredField("adj");
    adjField.setAccessible(true);
    @SuppressWarnings("unchecked")
    Map<Vertex, Map<Vertex, Edge<Vertex>>> adj =
        (Map<Vertex, Map<Vertex, Edge<Vertex>>>) adjField.get(g);

    Vertex v1Key = adj.keySet().stream().filter(x -> x.id() ==
1).findFirst().orElseThrow();
    Map<Vertex, Edge<Vertex>> nbrs = adj.get(v1Key);

    // Take some existing edge (1-2) and pretend it's a self-loop 1-1
in the adjacency map.
    Edge<Vertex> e = nbrs.values().iterator().next();
    nbrs.clear();
    nbrs.put(v1Key, e);  // now u == v in the first (and only)
inner-loop entry

    assertThrows(AssertionError.class, () -> invokeListCheckRep(g));
}
```

You can keep the three earlier list tests we already wrote:

- `listCheckRepMissingNeighbourVertex`
- `listCheckRepAsymmetricEdge`
- `listCheckRepEdgeNotIncidentOnEndpoint`

Together, they give both “passes” and “fails” paths for every assert in that method.

---

3. Matrix `checkRep` (second screenshot)

Your matrix `checkRep` is already in decent shape, and the two matrix tests I gave you:

- diagonal non-null
- asymmetric matrix

cover the juicy failure branches there. If you see any remaining orange for the very top asserts (`n >= 0`, `verticesByIdx.size() == n`, etc.), you *can* do the same reflection trick on `n` / `verticesByIdx` / `idx`, but in practice you’ll probably already be over 90% global branch coverage once the list `checkRep` is fully exercised.

---

4. Summary of what to do now

1. Make sure **JUnit is running with `-ea`**.
2. Add the test methods above to your existing `CheckRepBranchTests` class (they rely on the same `listGraph`, `v`, and `invokeListCheckRep` helpers you already have).
3. Re-run coverage.

You should now see:

- Nearly every assert in `AdjacencyListGraph.checkRep` with both branches hit.
- Matrix `checkRep` still mostly green thanks to the previous tests.

- Overall branch coverage for the project jumping up, very likely into/above that 90% target.

You said:
we need to specifically hit matrix graph at least 5 more lines

ChatGPT said:
Thought for 1m 19s

Nice, this is super precise. If matrix `checkRep` still has orange lines, it means some of its early asserts (*n, sizes, row lengths, etc.*) have only had their **true** branch executed, never the **false** one.

We'll fix that by adding **5 more evil tests**, each breaking a *different* assertion in `AdjacencyMatrixGraph.checkRep`.

> These go in the same `CheckRepBranchTests` class where you already have `invokeMatrixCheckRep(...)`.
> Make sure your JUnit config is still running with `-ea` (assertions enabled).

---

0. One small helper

Add this helper near the top of `CheckRepBranchTests`:

```
// Build a small but non-trivial adjacency-matrix graph.
private AdjacencyMatrixGraph<Vertex, Edge<Vertex>> simpleMatrixGraph()
{
    Set<Vertex> vertices = new HashSet<>();
    vertices.add(v(1));
    vertices.add(v(2));
    vertices.add(v(3));

    Set<Edge<Vertex>> edges = new HashSet<>();
    edges.add(new Edge<>(v(1), v(2), 1.0));
    edges.add(new Edge<>(v(2), v(3), 2.0));

    return new AdjacencyMatrixGraph<>(vertices, edges);
}
```

Also make sure `CheckRepBranchTests` imports `java.util.List;` and `java.util.Map;` (you already had `Map`).

---

1) Break `assert n >= 0`

```
/** Breaks n >= 0 by setting n to a negative value. */
@Test
public void matrixCheckRepNegativeN() throws Exception {
    AdjacencyMatrixGraph<Vertex, Edge<Vertex>> g =
simpleMatrixGraph();

    Field nField = AdjacencyMatrixGraph.class.getDeclaredField("n");
    nField.setAccessible(true);
    nField.setInt(g, -1);   // violate n >= 0

    assertThrows(AssertionError.class, () -> invokeMatrixCheckRep(g));
}
```

---

2) Break `assert verticesByIdx.size() == n`

```
/** Breaks verticesByIdx.size() == n by removing one vertex from the
list. */
@Test
public void matrixCheckRepVerticesByIdxSizeMismatch() throws Exception
{
    AdjacencyMatrixGraph<Vertex, Edge<Vertex>> g =
simpleMatrixGraph();

    // Grab n to keep it unchanged.
    Field nField = AdjacencyMatrixGraph.class.getDeclaredField("n");
    nField.setAccessible(true);
    int n = nField.getInt(g);

    Field vbiField =
AdjacencyMatrixGraph.class.getDeclaredField("verticesByIdx");
    vbiField.setAccessible(true);
    @SuppressWarnings("unchecked")
```

```
    List<Vertex> verticesByIdx = (List<Vertex>) vbiField.get(g);

    // Now verticesByIdx.size() == n - 1, but n itself stays the same.
    verticesByIdx.remove(verticesByIdx.size() - 1);

    assertEquals(n - 1, verticesByIdx.size());
    assertThrows(AssertionError.class, () -> invokeMatrixCheckRep(g));
}
```

---

3) Break `assert idx.size() == n`

```
/** Breaks idx.size() == n by removing one mapping from the idx map.
*/
@Test
public void matrixCheckRepIdxSizeMismatch() throws Exception {
    AdjacencyMatrixGraph<Vertex, Edge<Vertex>> g =
simpleMatrixGraph();

    Field nField = AdjacencyMatrixGraph.class.getDeclaredField("n");
    nField.setAccessible(true);
    int n = nField.getInt(g);

    Field idxField =
AdjacencyMatrixGraph.class.getDeclaredField("idx");
    idxField.setAccessible(true);
    @SuppressWarnings("unchecked")
    Map<Vertex, Integer> idx =
        (Map<Vertex, Integer>) idxField.get(g);

    //
```

You said:

hold up no more helpers just do it with the ones i have already made or type it out

ChatGPT said:

Thought for 1m 38s

Gotcha, matrix `checkRep` is the one still being stingy with coverage. Let's brute-force it. We'll **break different top-level asserts** in `AdjacencyMatrixGraph.checkRep` using reflection, one per test.

No new helpers; these just use:

- your existing `v(int)` helper
- and the `invokeMatrixCheckRep(...)` helper we already wrote earlier to call the private `checkRep()` and rethrow `AssertionError`.

If you *don't* have `invokeMatrixCheckRep` yet, you can copy it from my earlier message, but I won't redefine it here.

Add these **five tests** to your `CheckRepBranchTests` class.

---

1) Break `assert n >= 0`

```
// matrix: break n >= 0
@Test
public void matrixCheckRepNegativeN() throws Exception {
    // Build a small, valid matrix graph first
    Set<Vertex> vertices = new HashSet<>();
    vertices.add(v(1));
    vertices.add(v(2));

    Set<Edge<Vertex>> edges = new HashSet<>();
    edges.add(new Edge<>(v(1), v(2), 1.0));

    AdjacencyMatrixGraph<Vertex, Edge<Vertex>> g =
        new AdjacencyMatrixGraph<>(vertices, edges);

    // Set n < 0
    Field nField = AdjacencyMatrixGraph.class.getDeclaredField("n");
    nField.setAccessible(true);
    nField.setInt(g, -1);

    assertThrows(AssertionError.class, () -> invokeMatrixCheckRep(g));
```

```
}
```

---

2) Break `assert verticesByIdx.size() == n`

```
// matrix: break verticesByIdx.size() == n
@Test
public void matrixCheckRepVerticesByIdxSizeMismatch() throws Exception
{
    Set<Vertex> vertices = new HashSet<>();
    vertices.add(v(1));
    vertices.add(v(2));
    vertices.add(v(3));

    Set<Edge<Vertex>> edges = new HashSet<>();
    edges.add(new Edge<>(v(1), v(2), 1.0));
    edges.add(new Edge<>(v(2), v(3), 2.0));

    AdjacencyMatrixGraph<Vertex, Edge<Vertex>> g =
        new AdjacencyMatrixGraph<>(vertices, edges);

    // Get current n (keep it valid)
    Field nField = AdjacencyMatrixGraph.class.getDeclaredField("n");
    nField.setAccessible(true);
    int n = nField.getInt(g);

    // Shrink verticesByIdx by one element
    Field vbiField =
AdjacencyMatrixGraph.class.getDeclaredField("verticesByIdx");
    vbiField.setAccessible(true);
    @SuppressWarnings("unchecked")
    java.util.List<Vertex> verticesByIdx =
        (java.util.List<Vertex>) vbiField.get(g);

    verticesByIdx.remove(verticesByIdx.size() - 1);
    assertEquals(n - 1, verticesByIdx.size());

    assertThrows(AssertionError.class, () -> invokeMatrixCheckRep(g));
```

```
}
```

---

3) Break `assert idx.size() == n`

```
// matrix: break idx.size() == n
@Test
public void matrixCheckRepIdxSizeMismatch() throws Exception {
    Set<Vertex> vertices = new HashSet<>();
    vertices.add(v(1));
    vertices.add(v(2));
    vertices.add(v(3));

    Set<Edge<Vertex>> edges = new HashSet<>();
    edges.add(new Edge<>(v(1), v(2), 1.0));
    edges.add(new Edge<>(v(2), v(3), 2.0));

    AdjacencyMatrixGraph<Vertex, Edge<Vertex>> g =
        new AdjacencyMatrixGraph<>(vertices, edges);

    // n and verticesByIdx are consistent; break idx.size()
    Field idxField =
AdjacencyMatrixGraph.class.getDeclaredField("idx");
    idxField.setAccessible(true);
    @SuppressWarnings("unchecked")
    Map<Vertex, Integer> idx =
        (Map<Vertex, Integer>) idxField.get(g);

    // Remove one mapping
    Vertex someVertex = idx.keySet().iterator().next();
    idx.remove(someVertex);

    assertThrows(AssertionError.class, () -> invokeMatrixCheckRep(g));
}
```

---

4) Break `assert edgeMatrix.length == n`

```
// matrix: break edgeMatrix.length == n
@Test
public void matrixCheckRepMatrixLengthMismatch() throws Exception {
    Set<Vertex> vertices = new HashSet<>();
    vertices.add(v(1));
    vertices.add(v(2));
    vertices.add(v(3));

    Set<Edge<Vertex>> edges = new HashSet<>();
    edges.add(new Edge<>(v(1), v(2), 1.0));
    edges.add(new Edge<>(v(2), v(3), 2.0));

    AdjacencyMatrixGraph<Vertex, Edge<Vertex>> g =
        new AdjacencyMatrixGraph<>(vertices, edges);

    // Read n
    Field nField = AdjacencyMatrixGraph.class.getDeclaredField("n");
    nField.setAccessible(true);
    int n = nField.getInt(g);

    // Replace edgeMatrix with a shorter one
    Field matrixField =
AdjacencyMatrixGraph.class.getDeclaredField("edgeMatrix");
    matrixField.setAccessible(true);
    @SuppressWarnings("unchecked")
    Edge<Vertex>[][] oldMatrix =
        (Edge<Vertex>[][]) matrixField.get(g);

    // Shrink outer length by 1 (n-1)
    @SuppressWarnings("unchecked")
    Edge<Vertex>[][] newMatrix =
        java.util.Arrays.copyOf(oldMatrix, n - 1);

    matrixField.set(g, newMatrix);

    assertThrows(AssertionError.class, () -> invokeMatrixCheckRep(g));
}
```

---

5) Break `assert edgeMatrix[i].length == n` (row length)

```
// matrix: break edgeMatrix[i].length == n for some row
@Test
public void matrixCheckRepRowLengthMismatch() throws Exception {
    Set<Vertex> vertices = new HashSet<>();
    vertices.add(v(1));
    vertices.add(v(2));
    vertices.add(v(3));

    Set<Edge<Vertex>> edges = new HashSet<>();
    edges.add(new Edge<>(v(1), v(2), 1.0));
    edges.add(new Edge<>(v(2), v(3), 2.0));

    AdjacencyMatrixGraph<Vertex, Edge<Vertex>> g =
        new AdjacencyMatrixGraph<>(vertices, edges);

    // Read n and edgeMatrix
    Field nField = AdjacencyMatrixGraph.class.getDeclaredField("n");
    nField.setAccessible(true);
    int n = nField.getInt(g);

    Field matrixField =
AdjacencyMatrixGraph.class.getDeclaredField("edgeMatrix");
    matrixField.setAccessible(true);
    @SuppressWarnings("unchecked")
    Edge<Vertex>[][] matrix =
        (Edge<Vertex>[][]) matrixField.get(g);

    // Keep outer length correct (matrix.length == n),
    // but make row 0 shorter than n.
    @SuppressWarnings("unchecked")
    Edge<Vertex>[] shorterRow =
        java.util.Arrays.copyOf(matrix[0], n - 1);
    matrix[0] = shorterRow;

    assertThrows(AssertionError.class, () -> invokeMatrixCheckRep(g));
}
```

Each of these:

- Starts from a **valid** `AdjacencyMatrixGraph` (so earlier asserts are true).
- Then corrupts **one specific structural property** via reflection.
- Calls `checkRep()` (through `invokeMatrixCheckRep`), expecting an `AssertionError`.

Together with the two earlier matrix tests (diagonal non-null + asymmetric matrix), you'll be exercising:

- `n >= 0` (fail branch)
- `verticesByIdx.size() == n` (fail branch)
- `idx.size() == n` (fail branch)
- `edgeMatrix.length == n` (fail branch)
- `edgeMatrix[i].length == n` (fail branch)
- plus the symmetry + self-loop checks from the earlier tests.

That should easily get you the "5 more lines" and push matrix-`checkRep` branch coverage over the threshold.